zzrctsim: Design, Novel Features, and Distributional Scope

Ronald G. Thomas, Ph.D.

August 11, 2026

Contents

1 Purpose and scope

`zzrctsim` simulates randomized trials in order to evaluate design and analysis choices. It exists because 21 of the 35 research compendia under `~/prj/res` are simulation-dependent and 13 of them have no simulation code written yet; a shared engine introduced now prevents 21 divergent reimplementations rather than reconciling them later.

This report has three parts. Section 2 sets out the architecture and the contracts that hold it together. Sections 3 to 8 describe, in detail, the six features that are not available in existing packages. Sections 9 onward record the distributional scope: which response families are supported, why the exponential family unifies less than it appears to, which operations are Gaussian-only, and how the package refuses work it cannot do.

Every measured figure quoted below was produced by running the code, either in a chunk of this document or in the test suite, which currently holds 194 assertions.

2 Design

2.1 The layered pipeline

The package is organized so that the code mirrors ADEMP, and a manuscript's Methods subsections map onto modules rather than being reconstructed from a monolithic script.

Layer	Functions	Distribution-aware?
Design	`trial_schedule`, `runin_design`	no
Accrual	`accrue`, `close_out`, `realize_schedule`	no
DGM	`dgm_conditional`, `dgm_marginal`, `dgm_tte`, `cov_at`	yes
Generation	`generate_outcomes`, `generate_tte`	yes
Missingness	`dropout_mask`, `apply_mask`, `reference_based`	partly
Analysis	`fit_result`, `fit_ancova`	no
Driver	`run_simulation`, `sim_streams`	no

Layer	Functions	Distribution-aware?
Performance	`compute_performance`, `nsim_for_mcse`	no
Power	`sim_power`, `power_curve`, `sample_size`	no

Only three of nine layers know anything about the response distribution. That asymmetry is the central design fact: the reuse across endpoint families lives in the spine, and the variation lives in the generator. It is why the endpoint families are best described as *several generators behind one contract* rather than one generator with a family switch.

2.2 Three contracts

The DGM object answers `cov_at(dgm, times)` and nothing else is assumed of it. Because it is a function of time rather than a fixed matrix, a subject observed on an irregular grid, as staggered accrual produces, is handled without special-casing.

Complete data are generated first; missingness is a separate layer. This is what allows the information loss from dropout to be isolated by comparison against the complete-data run, and it is the only way a response-dependent mechanism can be expressed at all: a hazard that reads `y` requires `y` to exist first.

Fitters return one shape. `fit_result()` carries `estimate`, `se`, `p_value`, `ci_lower`, `ci_upper`, `df`, `converged`. The driver and the performance measures never inspect a model object, so a new analysis method is a function, not a change to the engine.

2.3 What is deliberately not here

The package does not derive group-sequential boundaries (`rpact`, `gsDesign` do), does not fit models beyond a reference ANCOVA (`mmrm`, `nlme`, `lme4`, `survival` are the engines), does not perform reference-based imputation for *analysis* (`rbmi` does), and is not a general Monte Carlo harness (`SimDesign` is). It simulates trials and measures what happens.

3 Novel feature 1: dual-parameterized data-generating mechanisms

A longitudinal DGM can be declared two ways: conditionally, through random effects `(Z, G, sigma^2)`, or marginally, through a covariance `V`. Existing packages pick one. `simr` and `lme4` are conditional only; `mmrm` and `nlme` are marginal only. `zzrctsim` accepts either and converts between them.

Forward conversion is exact and unremarkable: `V = Z G Z' + sigma^2 I`. Backward conversion is the interesting direction, because it is not always possible.

```
tt <- 1:6
cs  <- cov_cs(sd = 1, rho = 0.5)(tt)
```

```
ar1 <- cov_ar1(sd = 1, rho = 0.6)(tt)
c(CS = certify(cs)$q_min, AR1 = certify(ar1)$q_min)
```

```
##  CS AR1
##   1   5
```

Compound symmetry is reachable from a single random effect; AR(1) at six visits requires five. `as_conditional()` refuses an insufficient budget rather than silently returning an approximation.

```
as_conditional(dgm_marginal(cov_ar1(1, 0.6)), tt, q = 2)
```

```
## Error in `as_conditional()`:
## ! Target covariance is not reachable from 2 random effect(s): it requires q_min = 5.
##   Best attainable relative Frobenius error at q = 2 is 0.106.
##   Use `reach_error()` to inspect, or raise `q`.
```

This matters because the relationship between the two parameterizations is itself the subject of several compendia: 18 asks when random-effects models reduce to simpler forms, 04 uses the Woodbury identity to move between the two variance expressions, and 09 and 31 both turn on the same mapping.

4 Novel feature 2: the reachability certificate

The refusal above is not a heuristic. A random-effects model with `q` random effects and i.i.d. residuals induces `V = Z G Z' + sigma^2 I`, where `Z G Z'` is positive semi-definite of rank at most `q`. A target `V` is therefore reachable only if some `sigma^2 >= 0` makes `V - sigma^2 I` positive semi-definite with rank at most `q`.

Subtracting `sigma^2 I` shifts every eigenvalue equally, so the question is settled by the spectrum: take `sigma^2` to be the smallest eigenvalue and count what remains.

```
ct <- certify(ar1)
c(q_min = ct$q_min, sigma2 = round(ct$sigma2, 4),
  max_reconstruction_error = max(abs(reconstruct(ct) - ar1)))
```

```
##                    q_min                   sigma2 max_reconstruction_error
##                5.000e+00                2.654e-01                1.776e-15
```

The certificate is **constructive**: the eigenvectors supply `Z`, the shifted eigenvalues supply `G`, and the reconstruction is exact to machine precision.

`reach_error()` converts "unreachable" into a magnitude, which is what a practitioner actually needs:

```
knitr::kable(reach_error(ar1), digits = 4, booktabs = TRUE,
             caption = 'Best attainable approximation to AR(1) at each
                        random-effects budget.')
```

Table 1: Best attainable approximation to AR(1) at each random-effects budget.

q	sigma2	max_abs_error	rel_frobenius
1	0.6290	0.2982	0.2806
2	0.4363	0.1005	0.1059
3	0.3418	0.0348	0.0384
4	0.2922	0.0130	0.0114
5	0.2654	0.0000	0.0000

A single random effect is wrong by 28% in relative Frobenius norm. That is not a refinement of AR(1); it is a different covariance.

Two honest qualifications. First, `q_min` is **necessary but not sufficient for a particular model**: the certificate lets `Z` be the eigenvectors, whereas a real specification fixes `Z` to design columns such as an intercept and a slope. It proves impossibility cleanly and possibility only for an unconstrained `Z`. Second, the underlying linear algebra is classical – it is probabilistic PCA, equivalently factor analysis with isotropic uniquenesses – so the contribution is packaging it as an automatic check in a simulation workflow, not the mathematics.

5 Novel feature 3: trial phase structure and staggered accrual

No surveyed package has a vocabulary for the phases of a trial. `trial_schedule()` spans run-in, randomization, treatment and common close, with run-in times negative and randomization at zero.

```
sch <- trial_schedule(run_in = 2, treatment = 4, common_close = 2,
                      interval = 3)
knitr::kable(as.data.frame(sch), booktabs = TRUE,
             caption = 'A three-phase schedule.')
```

Table 2: A three-phase schedule.

index	time	phase	h	on_treatment
-2	-6	run_in	0	FALSE
-1	-3	run_in	0	FALSE
0	0	randomization	0	FALSE
1	3	treatment	1	TRUE
2	6	treatment	1	TRUE
3	9	treatment	1	TRUE
4	12	treatment	1	TRUE
5	15	common_close	1	FALSE
6	18	common_close	1	FALSE

Two design decisions are exposed rather than assumed.

The common-close convention. During the common close both arms are off treatment. Whether the treated arm reverts to the control trajectory or retains its accumulated benefit is a modeling choice that changes the estimand, so both are offered:

```
arm2 <- factor(c('placebo', 'active'), levels = c('placebo', 'active'))
rev <- runin_design(sch, arm2, common_close = 'revert')
ret <- runin_design(sch, arm2, common_close = 'retain')
rbind(revert = rev$x_trt_active[rev$arm == 'active'],
      retain = ret$x_trt_active[ret$arm == 'active'])
```

```
##        [,1] [,2] [,3] [,4] [,5] [,6] [,7] [,8] [,9]
## revert    0    0    0    3    6    9   12    0    0
## retain    0    0    0    3    6    9   12   12   12
```

The per-arm hinge. Each arm independently either continues its run-in slope through randomization or changes slope there. An unhinged control arm constrains `beta = delta`, dropping one parameter, and is what makes run-in observations directly informative about the control slope – the efficiency argument of compendium 04.

```
h  <- runin_design(sch, arm2, hinge = TRUE)
nh <- runin_design(sch, arm2,
                   hinge = c(placebo = FALSE, active = TRUE))
c(hinged_params = sum(grepl('^x_', names(h))),
  unhinged_params = sum(grepl('^x_', names(nh))))
```

```
##   hinged_params unhinged_params
##               3               2
```

The parameterization spans the same column space as the `delta`/`beta`/`gamma` form used in compendium 04, so the closed-form variance results derived there carry over unchanged.

Staggered accrual with a common close-out. Follow-up is not a constant. Subjects enrol over an accrual window and the study ends on one calendar date, so early enrollees are observed *longer* than the nominal schedule and the last enrollee exactly as long.

```
set.seed(3)
s2 <- trial_schedule(run_in = 1, treatment = 4, interval = 3)
enr <- accrue(5, period = 12, pattern = 'linear')
cl  <- close_out(enr, s2, rule = 'lslv')
rz  <- realize_schedule(s2, enr, cl, extend = TRUE)
data.frame(enrolled = enr,
           visits = as.integer(table(rz$id)),
           follow_up = as.numeric(tapply(rz$time, rz$id, max)))
```

```
##   enrolled visits follow_up
## 1        0     10        24
## 2        3      9        21
## 3        6      8        18
## 4        9      7        15
## 5       12      6        12
```

This is the mechanism that determines the information fraction at an interim analysis, and it is why `cov_at()` must be a function of time.

6 Novel feature 4: calibrated, mechanism-correct missingness

Dropout follows the selection-model factorization `f(Y, R) = f(Y) f(R | Y)`, with the discrete-time hazard of Diggle and Kenward (1994),

$$\text{logit}\, P(\text{drop at } j) = \psi_0 + \psi_1(y_{j-1} - c) + \psi_2(y_j - c).$$

Three properties distinguish this from the usual implementation.

The mechanisms are nested restrictions of one model, not three separate models: MCAR is `psi1 = psi2 = 0`, MAR is `psi2 = 0`, MNAR frees `psi2`. MNAR therefore *extends* MAR rather than replacing its history dependence.

The response is centred and `psi0` is calibrated to a target proportion, so the requested dropout is the realized dropout regardless of `psi1` and `psi2`. Because the hazards depend only on complete-data responses, the expected proportion has the closed form `1 - prod(1 - p_j)` averaged over subjects, so calibration is a one-dimensional root-find with no inner Monte Carlo.

```
set.seed(9)
og <- generate_outcomes(d, dgm_conditional(G = diag(c(9, 0.04)),
                                           sigma2 = 4),
                        beta = c(x_slope = 0.5, x_trt_active = -0.25),
                        intercept = 20)
cal <- do.call(rbind, lapply(list(c(0, 0), c(0.15, 0), c(0.15, 0.10)),
  function(ps) do.call(rbind, lapply(c(0.10, 0.25, 0.50), function(tg) {
    z <- dropout_mask(og, target = tg, psi1 = ps[1], psi2 = ps[2])
    sp <- attr(z, 'spec')
    data.frame(mechanism = sp$mechanism, psi1 = ps[1], psi2 = ps[2],
               target = tg, realized = as.numeric(sp$expected))
  })))
knitr::kable(cal, digits = 3, booktabs = TRUE, row.names = FALSE,
             caption = 'The target proportion is met across mechanisms
                        and across a range of psi1.')
```

Table 3: The target proportion is met across mechanisms and across a range of psi1.

mechanism	psi1	psi2	target	realized
MCAR	0.00	0.0	0.10	0.10
MCAR	0.00	0.0	0.25	0.25
MCAR	0.00	0.0	0.50	0.50
MAR	0.15	0.0	0.10	0.10
MAR	0.15	0.0	0.25	0.25
MAR	0.15	0.0	0.50	0.50

mechanism	psi1	psi2	target	realized
MNAR	0.15	0.1	0.10	0.10
MNAR	0.15	0.1	0.25	0.25
MNAR	0.15	0.1	0.50	0.50

Without centring, a hazard on an untransformed ADAS-Cog or CDR-SB scale saturates: `plogis(qlogis(0.12) + 0.25 * 20) = 0.95`, and every subject drops out at the first eligible visit while the nominal rate still reads 0.12.

Mask and filter are separate layers. `dropout_mask()` generates a pattern, `mask_from_data()` imports one from a completed trial, and `apply_mask()` applies either. A mask is therefore a first-class object that can be saved, inspected, and held fixed across replicates under common random numbers – none of which is possible when generation and application are fused in one call.

7 Novel feature 5: reference-based trajectories as a *generator*

`rbmi` implements reference-based assumptions for *analysis*. `zzrctsim` implements them as *generators*, so that data whose true post-discontinuation behaviour follows one assumption can be analysed under another and the resulting bias measured.

```
set.seed(11)
mkr <- dropout_mask(og, target = 0.40)
shifts <- vapply(c('j2r', 'cir', 'lmcf', 'cr'), function(m) {
  z <- reference_based(og, mkr, method = m)
  post <- z$discontinued & og$arm == 'active'
  mean(z$mu[post] - og$mu[post])
}, numeric(1))
round(shifts, 3)
```

```
##    j2r    cir   lmcf     cr
##  2.166  1.584 -1.584  2.166
```

The ordering `0 < CIR < J2R` is the expected one: copying increments in the reference retains the accumulated benefit that jumping to the reference discards. Each subject's realized residual is preserved, so the within-subject correlation survives the shift.

Discontinuation and missingness are deliberately separate operations, because under a treatment-policy estimand post-discontinuation values are observed and under a hypothetical estimand they are not. Fusing them would make the treatment-policy case, which ICH E9(R1) puts first, inexpressible.

8 Novel feature 6: Monte Carlo discipline as infrastructure

Morris, White and Crowther (2019) is followed in the code, not only in the write-up.

Streams, not seeds. `sim_streams()` produces `L'Ecuyer-CMRG` substreams. Independence is guaranteed by construction, and the same substreams replay across design cells, which is what common random numbers require; per-replicate `set.seed(i)` gives neither guarantee and desynchronizes silently when the amount of random-number consumption changes between cells.

Every replicate is reproducible in isolation. The driver stores the RNG state that produced each replicate, so a failing or anomalous one can be re-run alone for diagnosis rather than by re-running the study.

Errors are recorded, not propagated. A failure in generation or analysis marks the replicate non-converged, retains the message, and continues.

Every performance measure carries a Monte Carlo standard error.

```
set.seed(5)
theta <- -0.25 * max(s$time)
res <- run_simulation(
  B = 200,
  generate = function() generate_outcomes(d,
    dgm_conditional(G = diag(c(9, 0.04)), sigma2 = 4),
    beta = c(x_slope = 0.5, x_trt_active = -0.25),
    intercept = 20),
  analyse = list(ancova = function(z) fit_ancova(z, reference = 'placebo')),
  estimand = estimand('final-visit contrast', theta), seed = 7L)
perf <- compute_performance(res)
knitr::kable(perf[, c('measure', 'estimate', 'mcse')], digits = 4,
             booktabs = TRUE, row.names = FALSE,
             caption = 'Morris Table 6 measures, each with its Monte
                        Carlo standard error.')
```

Table 4: Morris Table 6 measures, each with its Monte Carlo standard error.

measure	estimate	mcse
n_converged	200.0000	NA
conv_rate	1.0000	NA
bias	0.0305	0.0203
emp_se	0.2873	0.0144
mod_se	0.2890	0.0006
rel_error_mod_se	0.5751	5.0455
mse	0.0831	0.0081
coverage	0.9600	0.0139
bias_elim_coverage	0.9700	0.0121
rejection	1.0000	0.0000
mean_ci_width	1.1345	0.0023

`nsim_for_mcse()` inverts these formulas so that the number of replicates is justified rather than conventional.

9 Sample size by simulation

`sample_size()` inverts a simulated power curve and then **confirms** the selected size with an independent longer run, reporting the achieved power and its Monte Carlo standard error. `simr`, the established comparator, has no inverse: the analyst reads a `powerCurve()` off by eye.

The practical value is that design features with no closed-form power formula are priced directly. The same design, with and without 30% MAR dropout:

```
set.seed(2)
sch3 <- trial_schedule(treatment = 4, interval = 3)
gd3 <- dgm_conditional(G = diag(c(9, 0.04)), sigma2 = 4)
b3 <- c(x_slope = 0.5, x_trt_active = -0.12)
dfn <- function(n) runin_design(sch3,
  factor(rep(c('placebo', 'active'), each = n),
         levels = c('placebo', 'active')), reference = 'placebo')
common <- list(design_fn = dfn, dgm = gd3, beta = b3, intercept = 20,
  estimand = estimand('final contrast', b3[['x_trt_active']] * max(sch3$time)),
  analyse = list(ancova = function(z) fit_ancova(z, reference = 'placebo')),
  B = 300L, seed = 5L)
ss_c <- do.call(sample_size, c(list(target = 0.80,
  n_grid = c(40, 70, 110, 160), confirm_B = 800L), common))
ss_d <- do.call(sample_size, c(list(target = 0.80,
  n_grid = c(60, 110, 160, 220), confirm_B = 800L,
  dropout = list(target = 0.30, psi1 = 0.10)), common))
data.frame(scenario = c('complete data', '30% MAR dropout'),
           n_per_arm = c(ss_c$n_per_arm, ss_d$n_per_arm),
           confirmed_power = c(ss_c$confirmation$power,
                               ss_d$confirmation$power),
           mcse = c(ss_c$confirmation$mcse, ss_d$confirmation$mcse))
```

```
##          scenario n_per_arm confirmed_power    mcse
## 1   complete data       108          0.8337 0.01316
## 2 30% MAR dropout       141          0.8275 0.01336
```

10 What the exponential family does and does not unify

The exponential family unifies *univariate marginal* distributions. It supplies no multivariate analogue: there is no canonical multivariate binomial or multivariate Poisson corresponding to the multivariate normal.

For a Gaussian response the marginal family and the dependence structure are the same object, the covariance matrix. For every other family they decouple, and a separate dependence mechanism must be chosen. That choice, and not the marginal law, is the design decision.

`zzrctsim` takes the conditional route. Random effects are drawn, `b_i ~ N(0, G)`, and the response is drawn conditionally, `y_ij ~ family(g^{-1}(eta_ij))` with `eta_ij = x_ij'beta`

`+ z_ij'b_i`. This is the GLMM construction used by `lme4` and `glmmTMB`.

Two consequences follow, and both belong to the user rather than to the package.

First, dependence between visits arises solely through the shared `b_i`. The reachable dependence is exactly what the random-effects structure can induce, and there is no marginal escape hatch: `dgm_marginal()` has no meaning outside the Gaussian case.

Second, under a non-identity link the marginal mean is not `g^{-1}(x'beta)`. The fixed effects are conditional on the random effects, so conditional and marginal treatment effects differ in magnitude. An `estimand()` used with a non-Gaussian family must state which of the two it names.

11 Supported families

```
s <- trial_schedule(treatment = 4, interval = 3)
arm <- factor(rep(c('placebo', 'active'), each = 300),
              levels = c('placebo', 'active'))
d <- runin_design(s, arm, reference = 'placebo')
ri <- function(t) matrix(1, length(t), 1)
b <- c(x_slope = 0.02, x_trt_active = -0.05)

specs <- list(
  list(nm = 'gaussian(identity)', f = gaussian(),         s2 = 4,    ic = 20),
  list(nm = 'binomial(logit)',    f = binomial(),         s2 = NULL, ic = 0),
  list(nm = 'binomial(probit)',   f = binomial('probit'), s2 = NULL, ic = 0),
  list(nm = 'poisson(log)',       f = poisson(),          s2 = NULL, ic = 1)
)

set.seed(8)
out <- do.call(rbind, lapply(specs, function(sp) {
  g <- dgm_conditional(G = matrix(0.5), sigma2 = sp$s2,
                       z = ri, family = sp$f)
  o <- generate_outcomes(d, g, beta = b, intercept = sp$ic)
  w <- matrix(o$y, ncol = nrow(s), byrow = TRUE)
  data.frame(family = sp$nm, min = min(o$y), max = max(o$y),
             mean = mean(o$y), within_cor = cor(w[, 2], w[, 3]))
}))
knitr::kable(out, digits = 3, booktabs = TRUE,
             caption = 'Response support and within-subject
                        dependence by family.')
```

Table 5: Response support and within-subject dependence by family.

family	min	max	mean	within_cor
gaussian(identity)	11.8	27.31	20.067	0.162
binomial(logit)	0.0	1.00	0.481	0.092

family	min	max	mean	within_cor
binomial(probit)	0.0	1.00	0.487	0.196
poisson(log)	0.0	38.00	3.417	0.663

The within-subject correlation is induced by the shared random intercept in every case. It is not a free parameter: it is whatever the random-effects structure implies on the response scale, and it differs by family even with `G` held fixed.

Negative binomial responses require `theta`, the dispersion in the `size` parameterization of `rnbinom()`. Binomial responses accept `trials`, defaulting to `1` for a binary outcome.

12 Time-to-event: single and recurrent

Time-to-event does not fit the covariance-based parameterization of the previous section, and this is a structural fact rather than an omission. Censoring, hazards and risk sets are a different kind of object from a marginal law, so `dgm_tte()` is a sibling generator behind the same contract rather than another `family` argument.

Generation is by **inversion of the cumulative hazard**, which is exact rather than a discretization of a grid, and works identically for both baseline hazards described below. Under proportional hazards the individual cumulative hazard is `H_i(t) = H_0(t) exp(lp_i)`, so if `E ~ Exp(1)` then `T = H_0^{-1}(E / exp(lp_i))` has exactly the intended distribution. For recurrent events the same inversion is applied to successive partial sums of i.i.d. `Exp(1)` variates, which yields the event times of a non-homogeneous Poisson process with intensity `h_i(t)`.

12.1 Single event

```
set.seed(4001)
subj <- data.frame(id = 1:4000, trt = rep(0:1, each = 2000))
g_wb <- dgm_tte(shape = 1.6, scale = 15)
g_wb
```

```
## <dgm_tte> time to first event
##   baseline: Weibull(shape =1.6, scale =15)
##   frailty SD: 0
```

```
ev <- generate_tte(subj, g_wb, beta = c(trt = log(0.65)),
                   followup = 30)
head(ev, 3)
```

```
##   id trt  time status
## 1  1   0 8.417      1
## 2  2   0 5.667      1
## 3  3   0 8.497      1
```

Administrative censoring is the natural companion to staggered accrual: passing `followup = close - enroll` censors every subject at the common close-out date, with follow-up decreasing in enrolment time.

```
fit_cox <- survival::coxph(survival::Surv(time, status) ~ trt,
                           data = ev)
c(true_log_HR = log(0.65),
  estimated = unname(coef(fit_cox)),
  se = sqrt(unname(vcov(fit_cox)[1, 1])),
  censored_fraction = mean(ev$status == 0))
```

```
##       true_log_HR         estimated                se censored_fraction
##          -0.43078          -0.49939           0.03381           0.09925
```

12.2 Recurrent events

Recurrent events are returned in counting-process format – one row per interval with `tstart`, `tstop`, `status` and `enum` – which is what `survival::coxph()` expects for an Andersen-Gill model. The intervals for a subject tile the follow-up window contiguously and end with a censored row.

```
set.seed(4002)
subj2 <- data.frame(id = 1:1500, trt = rep(0:1, each = 750))
g_rec <- dgm_tte(shape = 1, scale = 10, recurrent = TRUE)
rec <- generate_tte(subj2, g_rec, beta = c(trt = 0), followup = 20)
head(rec, 4)
```

```
##   id tstart  tstop status enum trt
## 1  1  0.000  8.353      1    1   0
## 2  1  8.353 20.000      0    2   0
## 3  2  0.000  2.750      1    1   0
## 4  2  2.750  3.228      1    2   0
```

A Gaussian **frailty** on the log-hazard scale induces within-subject dependence between gap times, which is what separates a recurrent process from independent replications. Its signature is overdispersion:

```
set.seed(4003)
counts <- function(sd) {
  g <- dgm_tte(shape = 1, scale = 10, recurrent = TRUE,
               frailty_sd = sd)
  r <- generate_tte(subj2, g, beta = c(trt = 0), followup = 20)
  unlist(tapply(r$status, r$id, sum))
}
data.frame(frailty_sd = c(0, 0.8),
           mean_events = c(mean(counts(0)), mean(counts(0.8))),
           var_events  = c(var(counts(0)),  var(counts(0.8))))
```

```
##   frailty_sd mean_events var_events
```

```
## 1        0.0       2.029      1.973
## 2        0.8       2.845      7.693
```

Without frailty the counts are Poisson, variance equal to mean. With frailty they are overdispersed. Note the consequence for interpretation: a frailty makes the *marginal* distribution non-proportional even though the conditional model is proportional hazards, which is the usual reason a fitted hazard ratio attenuates toward one. The estimand must say whether it names the conditional or the marginal hazard ratio.

12.3 Baseline hazards, and departures from proportionality

Two baseline hazards are available. The Weibull covers monotone hazards and reduces to the exponential at `shape = 1`. The **piecewise exponential** expresses an arbitrary hazard shape without committing to a parametric family, which is why it is the usual choice in trial simulation and what `simtrial` is built on. Its cumulative hazard is piecewise linear, so inversion remains exact.

```
g_pw <- dgm_tte(breaks = c(6, 12), rates = c(0.02, 0.06, 0.15))
g_pw
```

```
## <dgm_tte> time to first event
##   baseline: piecewise exponential, breaks [6, 12], rates [0.02, 0.06, 0.15]
##   frailty SD: 0
```

```
set.seed(4101)
sp <- data.frame(id = 1:40000, trt = 0)
e_pw <- generate_tte(sp, g_pw, beta = c(trt = 0), followup = 20)
emp <- function(lo, hi) {
  sum(e_pw$status == 1L & e_pw$time > lo & e_pw$time <= hi) /
    sum(pmax(0, pmin(e_pw$time, hi) - lo))
}
data.frame(interval = c('[0,6)', '[6,12)', '[12,20]'),
           specified = c(0.02, 0.06, 0.15),
           realized = c(emp(0, 6), emp(6, 12), emp(12, 20)))
```

```
##   interval specified realized
## 1    [0,6)      0.02  0.01965
## 2   [6,12)      0.06  0.05975
## 3  [12,20]      0.15  0.15071
```

A delayed treatment effect is the more consequential addition, because it breaks proportional hazards on purpose. Setting `effect_lag = L` gives intensity `h_i(t) = h_0(t) exp(lp_i * I(t > L))`: the arms are indistinguishable until `L` and diverge thereafter. The individual cumulative hazard remains invertible in closed form, so this is still generated exactly rather than by discretization.

This shape is standard in immuno-oncology and is what anti-amyloid trials in Alzheimer's disease are generally argued to show. A constant hazard ratio cannot represent it.

```
set.seed(4102)
s4 <- data.frame(id = 1:12000, trt = rep(0:1, each = 6000))
lag <- 8
e_lag <- generate_tte(s4, dgm_tte(shape = 1, scale = 20,
                                  effect_lag = lag),
                      beta = c(trt = log(0.4)), followup = 40)
fz <- survival::coxph(survival::Surv(time, status) ~ trt, data = e_lag)
data.frame(
  true_post_lag_HR = 0.4,
  fitted_constant_HR = exp(unname(coef(fz))),
  ph_test_p = survival::cox.zph(fz)$table[1, 'p'])
```

```
##   true_post_lag_HR fitted_constant_HR ph_test_p
## 1              0.4             0.6104 6.997e-76
```

The fitted constant hazard ratio is attenuated toward one relative to the true post-lag ratio of 0.4, and `cox.zph()` rejects proportionality. Both are the intended behaviour: the generator produces data that a proportional-hazards summary misrepresents, which is exactly what is needed to study how badly it misrepresents them.

12.4 An endpoint derived from the longitudinal trajectory

Compendium 13 compares MMRM against survival analysis. That comparison is only meaningful if both endpoints describe the *same* underlying process, rather than being drawn independently. `tte_from_trajectory()` defines the event as the first visit at which a generated trajectory crosses a threshold.

```
set.seed(4005)
s6 <- trial_schedule(treatment = 6, interval = 3)
arm6 <- factor(rep(c('placebo', 'active'), each = 300),
               levels = c('placebo', 'active'))
d6 <- runin_design(s6, arm6, reference = 'placebo')
lo <- generate_outcomes(d6, dgm_conditional(G = diag(c(4, 0.02)),
                                            sigma2 = 2),
                        beta = c(x_slope = 0.35, x_trt_active = -0.20),
                        intercept = 10)
evd <- tte_from_trajectory(lo, threshold = 16, direction = 'above')
a6 <- arm6[evd$id]
fit_d <- survival::coxph(survival::Surv(time, status) ~ a6, data = evd)
data.frame(arm = c('placebo', 'active'),
           crossed = c(mean(evd$status[a6 == 'placebo']),
                       mean(evd$status[a6 == 'active'])),
           mean_time_given_crossed = c(
             mean(evd$time[evd$status == 1 & a6 == 'placebo']),
             mean(evd$time[evd$status == 1 & a6 == 'active'])))
```

```
##       arm crossed mean_time_given_crossed
```

```
## 1 placebo    0.64                  12.36
## 2  active    0.25                  12.16
```

This table is worth pausing on. The treatment effect is large – 64% of placebo cross against 25% of active, a Cox hazard ratio of 0.289 – yet the mean crossing time *among those who crossed* is nearly identical between arms. Conditioning on having crossed selects the fastest progressors within each arm, and selects more severely in the arm where crossing is rarer. It is a compact illustration of why a survival endpoint is analysed through the hazard with censored observations retained, rather than through the mean event time of the subset that had events.

Subjects who never cross are censored at their last observed visit, so dropout and administrative censoring both propagate correctly into the survival endpoint.

13 Three scales, all retained

The GLMM path returns three columns rather than collapsing them, because for a non-identity link they are genuinely different quantities and a reader cannot reconstruct one from another without knowing the link.

```
set.seed(21)
gb <- dgm_conditional(G = matrix(0.5), z = ri, family = binomial())
ob <- generate_outcomes(d, gb, beta = b, intercept = 0)
knitr::kable(head(ob[, c('id', 'time', 'mu', 'eta', 'cmean', 'y')], 6),
             digits = 3, booktabs = TRUE,
             caption = 'mu is the fixed-effect linear predictor, eta
                        adds the random effects, cmean applies the
                        inverse link, y is the draw.')
```

Table 6: mu is the fixed-effect linear predictor, eta adds the random effects, cmean applies the inverse link, y is the draw.

id	time	mu	eta	cmean	y
1	0	0.00	0.561	0.637	1
1	3	0.06	0.621	0.650	1
1	6	0.12	0.681	0.664	0
1	9	0.18	0.741	0.677	0
1	12	0.24	0.801	0.690	1
2	0	0.00	0.369	0.591	0

The realized random effects are retained on a `ranef` attribute, so a replicate can be interrogated rather than only re-run.

14 What is Gaussian-only, and why

Four operations presuppose a Gaussian response. Each is now guarded.

14.1 Covariance of the response

`cov_at()` returns the covariance of the response. For a binomial or Poisson response the variance is determined by the mean, so it cannot be specified separately, and the function refuses.

```
cov_at(gb, s$time)
```

```
## Error in `cov_at.dgm_conditional()`:
## ! `cov_at()` is the covariance of the response and is defined only for the Gaussian famil
```

`linpred_cov()` is the defined alternative for every family: it returns `Z G Z'`, the covariance induced on the linear-predictor scale.

```
linpred_cov(gb, s$time)[1:3, 1:3]
```

```
##      [,1] [,2] [,3]
## [1,]  0.5  0.5  0.5
## [2,]  0.5  0.5  0.5
## [3,]  0.5  0.5  0.5
```

14.2 Conversion and the reachability certificate

`certify()`, `as_conditional()` and `as_marginal()` are results about `V = Z G Z' + sigma^2 I`. They are Gaussian statements and have no non-Gaussian analogue, because the marginal parameterization they convert to does not exist outside that case.

14.3 Reference-based post-discontinuation trajectories

`reference_based()` preserves each subject's additive residual `y - mu` while shifting the mean. On a count or binary scale that construction produces values outside the support: a probability shifted by a residual is not a probability, and a shifted count need not be an integer. The function refuses rather than returning impossible data.

```
mk <- dropout_mask(ob, target = 0.30)
reference_based(ob, mk, method = 'j2r')
```

```
## Error:
## ! `reference_based()` is defined only for a Gaussian response; this data set was generate
##   The construction preserves each subject's additive residual `y - mu`, which is not mean
```

14.4 Response-dependent missingness

The dropout hazard follows Diggle and Kenward (1994),

$$\operatorname{logit} P(\text{drop at } j) = \psi_0 + \psi_1 (y_{j-1} - c) + \psi_2 (y_j - c),$$

with the response centred at `c`. Two things are then scale-dependent: the default centring, the mean of `y`; and the interpretation of `psi1` as a change in log-odds per unit of `y`.

The guard is deliberately narrow. **MCAR is permitted for every family**, because its hazard never reads `y` at all. Only response-dependent missingness is restricted, and it is permitted for a non-Gaussian family as soon as `center` is supplied explicitly, since supplying it is the act of deliberation the guard exists to force.

```
guard <- function(label, expr) {
  r <- tryCatch({ expr; 'allowed' },
                error = function(e) 'blocked')
  data.frame(call = label, result = r)
}
knitr::kable(rbind(
  guard('binomial, MCAR',              dropout_mask(ob, target = 0.2)),
  guard('binomial, MAR, no center',    dropout_mask(ob, target = 0.2,
                                                    psi1 = 0.3)),
  guard('binomial, MAR, center given', dropout_mask(ob, target = 0.2,
                                                    psi1 = 0.3,
                                                    center = 0.5)),
  guard('binomial, reference_based',   reference_based(ob, mk, 'j2r'))
), booktabs = TRUE, row.names = FALSE,
   caption = 'Guards permit MCAR for any family and require an
              explicit centring for response-dependent mechanisms.')
```

Table 7: Guards permit MCAR for any family and require an explicit centring for response-dependent mechanisms.

call	result
binomial, MCAR	allowed
binomial, MAR, no center	blocked
binomial, MAR, center given	allowed
binomial, reference_based	blocked

15 What is family-agnostic

The guards are narrow because most of the package makes no distributional assumption at all. The design, missingness-application, driver and performance layers operate on estimates and indices, not on a response distribution:

Layer	Functions	Family-agnostic
Schedule	`trial_schedule`, `runin_design`	yes
Accrual	`accrue`, `close_out`, `realize_schedule`	yes
Mask application	`apply_mask`, `mask_from_data`	yes

Layer	Functions	Family-agnostic
Driver	`run_simulation`, `sim_streams`, `with_rng_state`	yes
Performance	`compute_performance`, `nsim_for_mcse`	yes
Power	`sim_power`, `power_curve`, `sample_size`	yes
Generation	`generate_outcomes`, `generate_tte`	family-aware
Covariance	`cov_at`, `certify`, `as_conditional`	Gaussian only
Mask generation	`dropout_mask`, `apply_dropout`	Gaussian, or explicit centring
Pattern mixture	`reference_based`	Gaussian only

This is why the endpoint families of the design document are described as *five generators behind one contract* rather than one generator with five settings. The reuse lives in the spine, which is built; the variation lives in the generator, which is not yet complete.

The spine carries a non-Gaussian study end to end today:

```
fit_glm <- function(z) {
  zz <- z[abs(z$time - max(z$time)) < 1e-8, ]
  m <- tryCatch(glm(y ~ arm, data = zz, family = binomial()),
                error = function(e) NULL)
  if (is.null(m)) return(null_fit())
  cf <- summary(m)$coefficients
  fit_result(cf[2, 'Estimate'], cf[2, 'Std. Error'], cf[2, 'Pr(>|z|)'])
}
res <- run_simulation(
  B = 300,
  generate = function() generate_outcomes(d, gb, beta = b, intercept = 0),
  analyse = list(glm = fit_glm),
  estimand = estimand('conditional log-OR at final visit', NA_real_),
  seed = 2L
)
perf <- compute_performance(res, theta = 0)
knitr::kable(perf[perf$measure %in% c('n_converged', 'emp_se',
                                      'mod_se', 'rejection'),
                  c('measure', 'estimate', 'mcse')],
             digits = 4, booktabs = TRUE, row.names = FALSE,
             caption = 'Morris Table 6 measures for a binary endpoint.')
```

Table 8: Morris Table 6 measures for a binary endpoint.

measure	estimate	mcse
n_converged	300.0000	NA
emp_se	0.1648	0.0067
mod_se	0.1652	0.0001
rejection	0.9100	0.0165

Note that `theta` is passed as `0` for a null check rather than as a marginal log-odds ratio. The estimand under a logit link is ambiguous between the conditional and marginal effect, and the package does not resolve that ambiguity on the user's behalf.

16 Boundaries and open work

Time-to-event, added above, is one of the two families that fall outside the exponential family and needs its own generator rather than another `family` argument. One remains.

Mixtures of alpha-stable laws. No closed-form density, and for stability index below two no finite variance. That last point reaches beyond the generator: without moments, bias, empirical standard error and mean squared error are undefined, so `compute_performance()` must dispatch on whether moments exist rather than return a finite-looking number. This is the one place where an endpoint family breaks the family-agnostic spine.

A copula route would allow arbitrary marginals with a specified correlation, but the requested correlation is not the achieved one for discrete responses, and the shortfall worsens as events become rarer. If that route is added, the achieved correlation must be computed and returned rather than assumed.

17 Reproducibility

```
## zzrctsim: development version

## R: R version 4.6.1 (2026-06-24)
```

Rendered on 2026-08-11 at 18:48 PDT.
Source: ~/prj/sfw/17-zzrctsim/zzrctsim/analysis/report/report.Rmd